内核设计优化实现文档

BuildStorm 场景下 MossOS 全链路优化

参赛院校： 重庆邮电大学
参赛队名： 每天睡满八小时
队伍成员： 吴勋、张可欣
指导教师： 马彬、钟福金

2026 年 **8** 月

目录

1 概述 1
2 问题定位与根因分析 2
2.1 负载特征 2
2.2 火焰图定位问题 2
2.3 与 Linux 基线的同口径对比 3
2.4 根因分析 4
3 优化设计与实现 8
3.1 优化一：per-CPU 物理页缓存（PCP） 8
3.2 优化二：AddrSpace 三段式 ——“prepare/execute/commit” 9
3.3 优化三：Folio 系列：folio 页缓存与只读文件页直接映射 12
3.4 优化四：ext4 锁外数据读写 + VirtIO 多在途请求 15
3.5 优化五：mprotect 按 VMA 定位并批量修改权限 18
3.6 优化六：地址空间回收与 TLB 刷新 20
3.7 其他优化 22
4 实验分析 24
4.1 实验环境与方法 24
4.2 性能数据展示 24
4.3 逐项分析 27
4.4 小结 28
5 AI 使用说明与可复现步骤 29
5.1 AI 使用说明 29
5.2 可复现步骤 29
6 总结与展望 30

第 1 章 概述

BuildStorm 题目要求内核在 QEMU 模拟的 RISC-V64 与 LoongArch64 平台（`-smp 8 -m 8G`）上离线从源码全量编译 `arceos-helloworld`，这是一次数百 crate 的真实 Rust 编译，伴随重度文件读写、多线程与大量 `mmap`/`mprotect` 系统调用。评分上，编译成功为门槛，加速分按编译耗时与同一评测机 Linux 基线 `B` 的关系计算，因此能否完整跑通并显著快于两倍 Linux 基线直接决定本题得分。

针对这一目标，我们首先在带 guest profiler 的内核上完整运行 BuildStorm，通过火焰图与路径探针定位问题与根因。结果显示瓶颈集中在文件缺页、页缓存与文件系统 I/O 链路，主要问题包括：

- 缺页处理持有地址空间独占锁等待磁盘 I/O；
- 物理页分配与释放集中在全局分配器；
- 只读文件映射多做一次私有页分配与复制；
- 页缓存按单页组织，逐页查找、重复读盘；
- ext4 全局锁内等待块设备；
- `mprotect` 反复扫描全局映射记录。

这些全局锁、逐页处理和重复工作使 8 个 vCPU 的并行能力无法发挥（优化前 6～8 核时间占比仅 6.23%）。围绕上述根因，我们完成了以下关键优化：

- per-CPU 物理页缓存（PCP），缓解全局分配器竞争；
- AddrSpace 三段式缺页（prepare / execute / commit），慢操作移出地址空间锁；
- folio 页缓存，合并相邻页的加载与 I/O；
- 只读文件页直接映射，真正写入时才 COW；
- ext4 数据 I/O 移出全局锁，配合异步 VirtIO 多在途请求；
- per-VMA `mprotect`，只更新目标区间；
- 后续补充：VFS 目录与 inode 元数据并发、缺页提交微路径、异步地址空间回收与 TLB 合并刷新、精确 TLB shootdown。

实验数据均为本地测试所得（采用 QEMU `virt`、8 vCPU、8 GiB、同一磁盘镜像、关闭 guest profiler），受本地电脑配置限制，实验数据会与测评机实际配置存在偏差，本地数据主要用于横向比较各优化阶段的效果。在此口径下，优化前 BuildStorm 正式编译耗时为 1524.50 秒；六项核心优化完成后降至 628.53 秒（-58.77%，约 2.43 倍加速）；继续完成后续优化后进一步降至 460.76 秒（累计 −69.78%）。平均有效核数由 3.252 核提高到 4.410 核，6～8 核时间占比由 6.23% 提高到 42.93%。

目前，Moss 内核已在评测机上完成 BuildStorm 测例的正式测评，RISC-V64 与 LoongArch64 两侧得分均为 **180** 分。排行榜截图如下：

#	用户名	队伍	学校	提交次数 (ASC)	buildstorm-glibc-la	buildstorm-glibc-rv	cagent-glibc-la	cagent-glibc-rv	rank
1	T202610617999569	每天睡满八小时	重庆邮电大学	44	180.0	180.0	199.1	199.1	758.2000
2	T202612332999758	这行代码有问题	烟台南山学院	76	180.0	180.0	199.1	199.1	758.2000
3	T2026104869910383	洗剪吹	武汉大学	73	180.0	179.0	199.1	199.1	757.2000
4	T2026181239910993	Unicus	哈尔滨工业大学（深圳）	17	180.0	150.6	199.1	199.1	728.8000

图 1　BuildStorm 测评成绩截图（截至 2026.8.5）

第 2 章 问题定位与根因分析

2.1 负载特征

BuildStorm 的正式编译任务由构建助手 `tg-xtask` 驱动，先进行 446 个 crate 的预构建，再执行正式目标构建。从内核视角看，这一负载有以下特征：

- 小文件密集：数百个 crate 源码、`.rlib`、增量产物和临时文件，大量文件以 `MAP_PRIVATE` 方式映射并顺序读取，单文件多为 4 KiB 页的整数倍；
- 多线程高并发：cargo/rustc 并行调度，8 个 vCPU 上同时存在多个编译任务与页缺失；
- 进程生命周期短：rustc、链接器、构建脚本频繁 fork/exec/exit，地址空间反复建立与销毁，exec 需要整体切换地址空间；
- 内存映射系统调用频繁：`mmap/munmap/mprotect` 被大量调用，其中 `mprotect` 尤其密集；
- 文件系统 **I/O** 重：冷缓存时大量页缓存 miss 需要进入 ext4 查询块映射并等待块设备。

这些特征叠加在一起，使得“单次系统调用本身的耗时”并不重要，重要的是缺页、页缓存、文件系统与内存管理路径上的全局锁竞争和串行等待被放大。

2.2 火焰图定位问题

我们在带 guest profiler 的内核上完整运行 BuildStorm，对每个 vCPU 以 10 Hz 采样内核/用户栈，并对 mutex 等待、ext4 全局锁等待、页缓存与同步块 I/O 等事件挂接探针，生成 CPU 火焰图与等待类火焰图，分别反映时间的消耗热点与线程的阻塞热点。

需要说明的是，这一火焰图方法并非仅在优化前运行一次：每完成一轮关键优化后，我们均使用相同的 profiler 配置重新采集火焰图，通过前后对比来判断优化是否命中了目标热点、瓶颈是否发生了迁移。报告中引用的数值如无特别说明，均来自优化前基线（`riscv-buildstorm-guest-20260721`）；优化后的对比数据来自 2026-07-24 整包优化后的复测（`riscv-buildstorm-guest-20260724`），两份报告的实验配置与采样口径完全一致，具备同口径可比性。

赛题只计时 `cargo xtask arceos build -p arceos-helloworld`（即实验中的 timed 阶段，`elapsed_s`），因此分析聚焦 timed 阶段。下图是优化前基线的 timed 阶段的 CPU 火焰图：

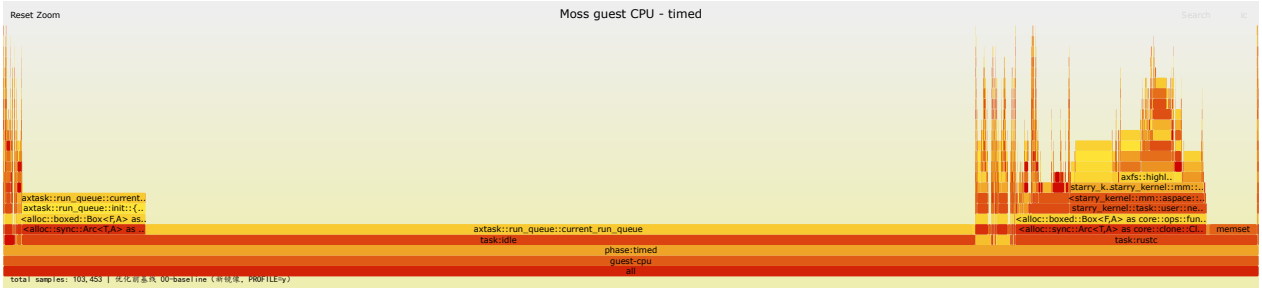


图 2　优化前 timed 阶段 CPU 火焰图

从 CPU 火焰图可以发现以下主要问题：

- 并行度低：`task:idle` 占全部采样的 75.9%，非 idle 仅 24.1%；
- 内核时间集中在缺页路径：最宽的塔是 `BackendOps::populate ->CowBackend::alloc_new_at -> alloc_new_frame -> alloc_frame`，其上方还挂着 `with_page_or_insert_impl`、`page_or_insert`、`PageCache::new` 等页缓存函数；
- 锁/关中断/清零开销明显：叶函数热点见下表。

热点	样本数	占比
NoPreemptIrqSave::release	10252	41.19%
lwext4 memset	4748	19.07%
virtio VirtQueue::can_pop	1137	4.57%
core::ptr::read	749	3.01%
BTreeMap navigate	500	2.01%
disable_preempt	450	1.81%
MemorySet::protect	230	0.92%

表 2-1　优化前 timed 阶段主要 CPU 热点（叶函数，占非 idle 样本）

mutex 等待火焰图如下，最宽的塔是 `task:rustc -> new_user_task -> BackendOps::populate -> CowBackend::alloc_new_at`：rustc、lto、opt 等编译任务经这条 COW/文件缺页路径的等待合计约 89%。也就是说，缺页线程在持有进程 `AddrSpace` mutex 时进入文件读取，再被页缓存与 ext4 的 mutex 挡住，层层排队。

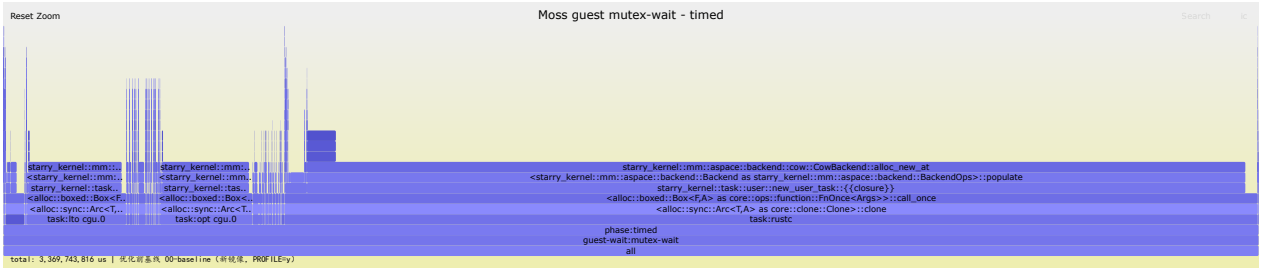


图 3　优化前 timed 阶段 mutex 等待火焰图

页缓存与文件系统的时间去向由事件探针给出：

事件	累计值
page-cache inclusive	4042.25 s
mutex wait	3369.74 s
ext4 inclusive	453.85 s
精确 ext4 全局锁等待	103.89 s
同步块读取	170.38 s
同步块写入	6.23 s
all off-CPU	70132.78 s

表 2-2　优化前 timed 阶段路径累计值（跨任务累计，可大于墙钟）

其中 `page-cache inclusive` 占据主导（单个 rustc 缺页栈占 70.82%，各任务合计约 84%），最宽路径仍是缺页；ext4 全局锁等待 103.89 s 中最大的单一路径同样是缺页读取（79.16%）。CPU、mutex 等待与页缓存/ext4 三组数据交叉指向同一条热点链：”缺页 → `AddrSpace` 大锁 → 页缓存 → ext4 → 块设备”，具体性能热点在下一节逐项展开。

2.3　与 Linux 基线的同口径对比

为了更清晰直观的看原来的 Moss OS 的性能缺陷，我们以相同事件模型对 Linux guest 重新采样，得到了当时的对照：

指标	**Linux**	**Moss**（优化前）	差距
正式阶段	454.22 s	2524.89 s	约 5.56 倍
锁等待（预构建/正式）	1.503 / 1.251 s	11166.94 / 8172.23 s	约 6500～7400 倍
page-cache inclusive（预构建/正式）	53.57 / 22.00 s	15042.35 / 9343.23 s	约 280～425 倍
前台同步 I/O（预构建/正式）	6.386 / 2.673 s	456.66 / 187.39 s	约 70 倍
ext4 read/write（预构建/正式）	70.05 / 22.49 s	2222.89 / 682.54 s	约 30 倍

表 2-3　Moss（优化前）与 Linux 的 BuildStorm 同口径对比

Linux 的活跃核等价值为 6.20 / 5.48 核，而优化前 Moss 只有 3.68 / 2.36 核。通过对比可以说明 Moss 与 Linux 的差距主要集中在文件缺页链路（锁等待、page-cache、同步 I/O、ext4）而非编译器本身的执行效率，首要修复方向是拆除“AddrSpace → CowBackend → page-cache → ext4 → block”的跨层大临界区。

2.4　根因分析

在逐条分析热点之前，下图展示了优化前发生缺页的完整处理链路。图中左右两侧四根彩色竖线分别标注四种锁的持锁周期——AddrSpace 独占锁、cache_op + page_cache 锁、ext4 全局自旋锁、全局页分配器锁。

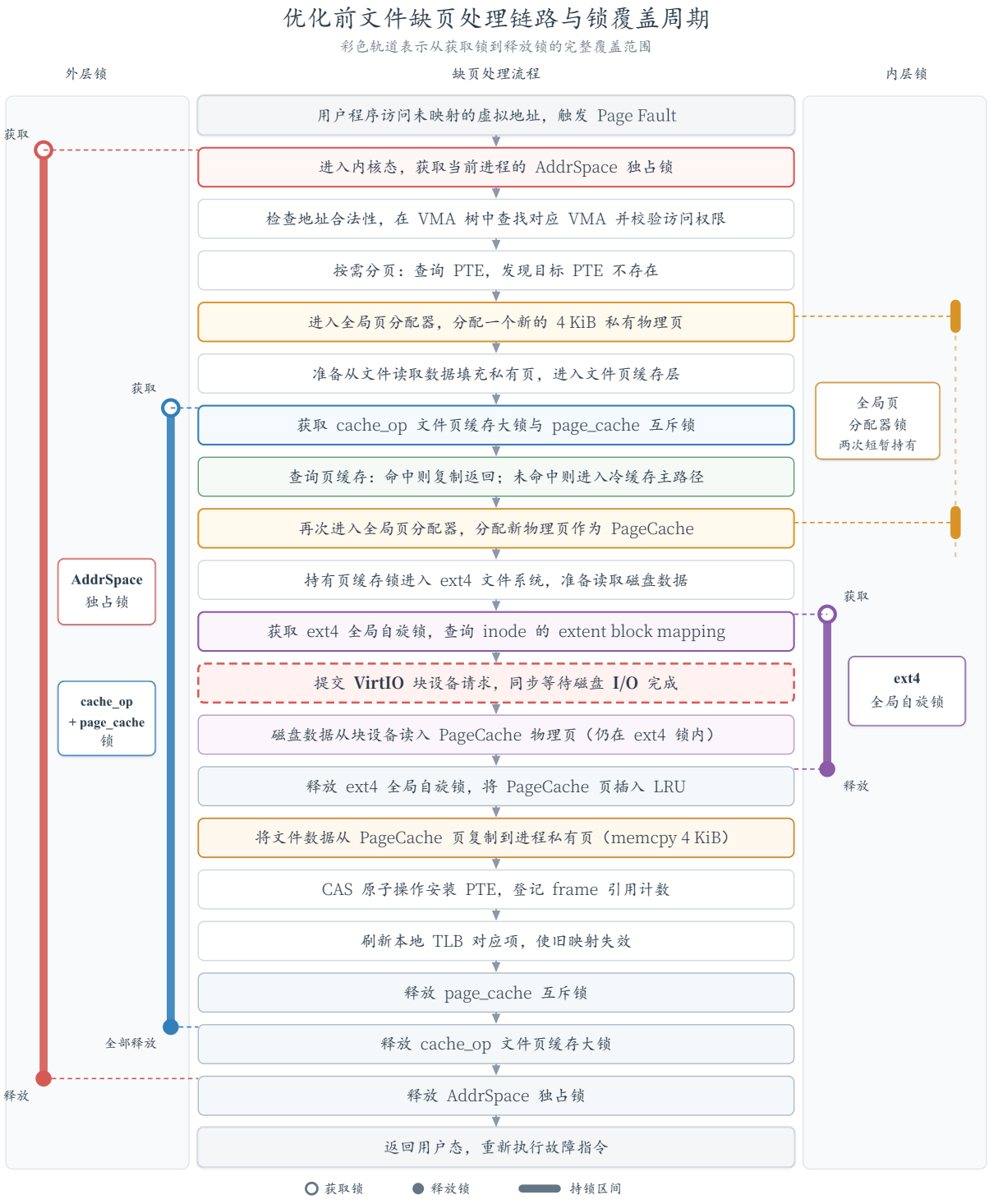


图 4　优化前文件缺页处理链路与锁范围

从图中就可以清晰地看到旧链路的两类结构性问题。

一方面是“大锁套小锁”：外层的 AddrSpace 锁尚未释放就逐层获取 page_cache 锁和 ext4 锁，内层锁上的任何阻塞都会直接穿透到外层的所有等待者。当一个线程在 ext4 锁内等磁盘 I/O 的时候，不仅持有 ext4 锁不放，还同时卡住了 page_cache 锁和 AddrSpace 锁，其他 7 个核上所有需要这些锁的线程就全部都要排队。

另一方面是“锁粒度太大”：同一把锁将快操作和慢操作捆在一起——ext4 锁内既做微秒级的 extent 映射查询，也做毫秒级的同步磁盘等待；AddrSpace 锁内既有轻量的 VMA 查找，也有重量级的物理页分配和文件数据复制。一些不需要锁内进行的重任务也被纳入了锁内。

并且两类问题会互相放大：内层锁做着毫秒级的慢活，外层锁只能被动等待，而 BuildStorm 会同时运行成百上千个 rustc/cc1 进程、每个都在反复缺页读文件，8 个 vCPU 的并行能力被层层锁链压缩为实测仅 2.36 核的有效并行度，导致多核被串行化，难以发挥多核的效率。优化前 mutex 累计等待高达 8172.23 s（跨任务累计），其中约 74% 经过图中所绘的这条缺页锁链。

因此，后面将介绍的六项核心优化的本质方向是一致的：缩短持锁时间（三段式、ext4 锁外 I/O）、降低进锁频率（PCP、folio 合并）、缩小锁的管辖范围（per-VMA mprotect），每一项都在拆除这条锁链中的一环。

结合火焰图、探针累计值与代码阅读，将主要瓶颈归纳为以下六个根因：

2.4.1 根因一：缺页处理持有整个地址空间的独占锁等待慢操作

在旧链路中，一次文件缺页会独占当前进程的 `AddrSpace`，在锁内完成 VMA 查找、物理页分配、页缓存 miss、ext4 块映射查询、磁盘 I/O 等待、数据复制与 PTE 安装。磁盘 I/O 的等待时间远大于 VMA 查询本身；一个线程等盘时，其他线程即使缺的是完全不同的文件或地址，也只能排队。BuildStorm 线程多、文件多，把这种串行等待放大是最主要的性能瓶颈。优化前火焰图中，约 74% 的 mutex 累计等待（8172.23 s，timed 阶段）会经过 `BackendOps::populate → CowBackend::alloc_new_at` 这条 COW 文件缺页路径。

2.4.2 根因二：物理页分配与释放集中在全局分配器

缺页分配私有页、页缓存分配页、COW 分配页以及进程退出时释放页，都会进入全局物理页分配器并竞争同一把锁。8 个 vCPU 同时缺页时，所有 CPU 的分配和释放都被串行化。优化前 CPU 火焰图中 `NoPreemptIrqSave::release` 约占非 idle 样本的四成以上，是排名第一的叶函数热点。

2.4.3 根因三：MAP_PRIVATE 只读缺页多一次私有页分配与复制

对于只读使用的 `MAP_PRIVATE` 文件映射，旧实现即使只是读取，也会”先分配页缓存页、读盘、再为进程分配一张私有物理页、从页缓存复制 4 KiB、安装私有页 PTE”。大量只读文件页永远不会被写，私有页分配和 4 KiB 复制完全浪费，且每个进程映射同一文件都会各复制一份，加剧内存压力与缺页成本。这一问题主要来自代码逻辑分析——火焰图中 `CowBackend::alloc_new_at` 是最宽的缺页塔，而其中的 memcpy 和私有页分配正是该塔的组成部分，但火焰图本身无法直接区分”必要的复制”与”浪费的复制”。

2.4.4 根因四：页缓存粒度小，逐页查找、重复 I/O 与重复等待

旧实现页缓存按单页组织：一次 miss 只加载一张 4 KiB 页，相邻页需要再次进入页缓存查询并可能再次读盘；多个线程同时缺同一页时还可能重复发起 I/O。优化前探针显示 page-cache inclusive 累计 9343.23 s（timed 阶段），是量级最大的嵌套路径，火焰图中占据约 91.5% 的图宽，其中 71.85% 经过 `with_page_or_insert_impl`。

2.4.5 根因五：ext4 全局锁内等待块设备

Moss 用一把全局锁保护 `LwExt4Filesystem` 元数据。旧实现中一次文件读取在锁内完成块映射查询后继续等待磁盘 I/O 完成才释放锁，导致不同文件的数据读取被全局串行化。优化前探针显示 ext4 全局锁等待累计 218.92 s（timed 阶段），同步块读取 175.67 s；ext4 锁等待火焰图中最大的单一调用栈同样是 rustc 文件映射缺页（占 64.80%）。

2.4.6 根因六：mprotect 扫描全局映射记录与无关 VMA

旧实现维护所有进程共享的 `MMAP_RECORDS` 全局数组。一次 `mprotect` 需要扫描全局记录检查 seal/最大权限、修改页表后再次锁全局记录并重建数组；`MemorySet::protect` 则从地址空间头部遍历所有 VMA，而调用实际只覆盖一小段地址。旧插桩显示平均一次调用扫描约 3.16 万条全局记录，预构建阶段累计扫描约 897 亿条。

需要指出的是，在优化前火焰图中 mprotect 并非可见的主热点——此时缺页锁链造成的阻塞量级远大于全局记录扫描（`mprotect` 相关 mutex 等待仅占 1.18%），将其完全掩盖。这一问题由代码插

桩而非火焰图发现。下图是缺页链路优化后重新采集的 mutex 等待火焰图，可以看到 mprotect 已成为压倒性的第一热点——占 mutex 累计等待的 95.16%、非 idle CPU 样本的 56.82%，是典型的“被更大瓶颈掩盖的独立串行化源”。

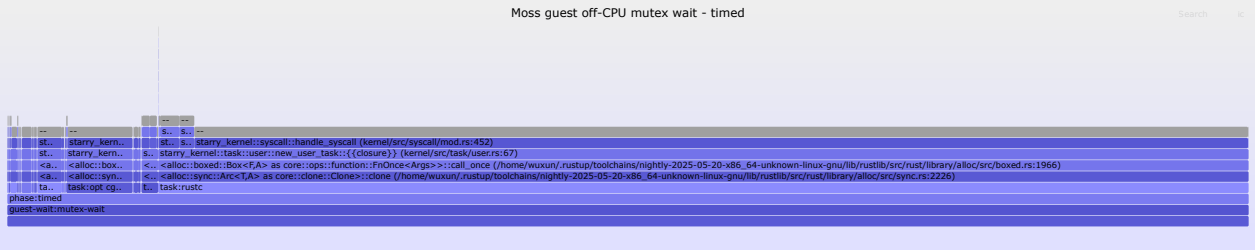


图 5　缺页链路优化后 timed 阶段 mutex 等待火焰图

第 3 章 优化设计与实现

本章按“六项核心优化 + 其他优化”两部分组织，六项核心优化按依赖顺序实施，其余针对次生热点的改进归入“其他优化”。每项优化均遵循同样的工程约束：修改独立提交、保留原始测量数据、先通过正确性测试再评估性能，避免把多项修改打包后无法归因。实现提交与实验数据的对应关系见第 4 章。

3.1 优化一：per-CPU 物理页缓存（PCP）

3.1.1 设计思路

如 2.4 根因所述，所有 order-0（4 KiB）物理页的分配与释放都集中在全局分配器，8 个 vCPU 并发缺页时全局锁成为热点（火焰图中 `NoPreemptIrqSave::release` 占非 idle 采样的 41.19%）。

为此在每个 CPU 身边维护一小撮空闲 4 KiB 物理页（PCP, Per CPU Pagecache），分配和释放优先命中本地缓存，仅在本地未命中或缓存空/满时才批量访问全局分配器。进行物理页分配时，若 PCP 缓存中没有，则批量从全局分配器补充，放入 PCP；释放物理页时，先放回 PCP，缓存过多时再批量归还全局分配器。

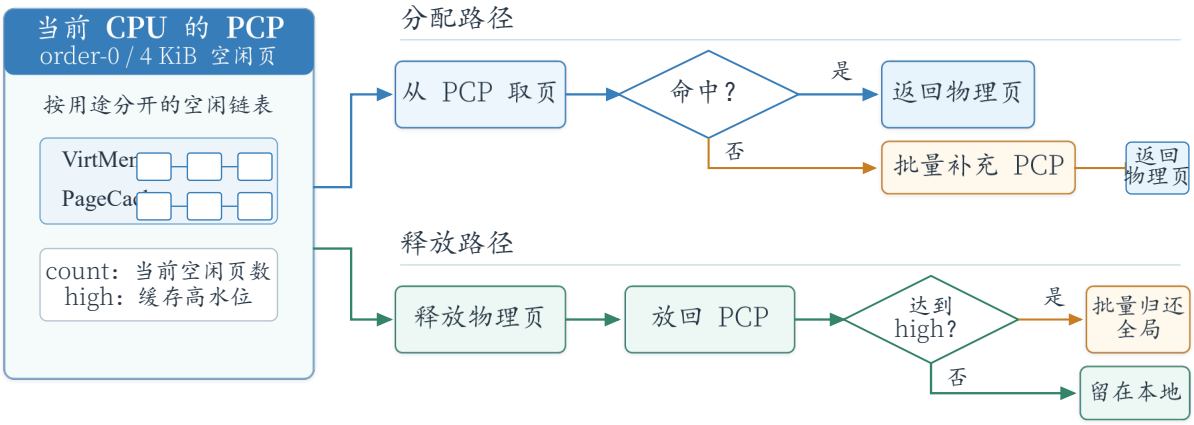


图 6　PCP 物理页缓存：分配与释放流程

3.1.2 实现要点

核心数据结构 `PerCpuPageCache` 为每种 `UsageKind` 维护一条独立空闲链表，通过 `cpu_id % 64` 定位槽位，`batch` 和 `high` 两个参数控制批量进出粒度：

```
pub(crate) struct PerCpuPageCache {
    // 每种 UsageKind（VirtMem / PageCache / ...）一条空闲链表
    heads: [usize; UsageKind::VARIANTS.len()],
    kind_counts: [usize; UsageKind::VARIANTS.len()],
    count: usize,           // 当前缓存总页数
    batch: usize,           // 每次 refill / drain 的批量页数
    high: usize,            // 当前高水位线
    high_min: usize,        // 高水位下限（8 GiB / 8核 ≈ 624）
    high_max: usize,        // 高水位上限（≈ 32768）
    // ...
}
```

分配路径：快路径仅持本 CPU 槽位的 IRQ 锁，命中时直接从链表头部弹出；未命中再走慢路径加全局分配器锁批量补货。

```rust
pub fn alloc_page_cached(&self, cpu_id: usize, cpu_count: usize,
                         kind: UsageKind) -> AllocResult<usize> {
    let slot = cpu_id % PCP_CPU_SLOTS;
    // 快路径：只锁本 CPU 槽位，IRQ 级别即可
    let mut cache = self.page_caches[slot].lock();
    if let Some(page) = cache.try_allocate(total_pages, cpu_count, kind) {
        return Ok(page);   // 命中，直接返回，全程不碰全局锁
    }
    drop(cache);
    // 慢路径：关抢占 + 全局分配器锁，批量 refill batch 页
    self.alloc_page_cached_slow(slot, total_pages, cpu_count, kind)
}
```

释放路径：先推回本 CPU 的 PCP；当缓存页数超过高水位时，批量退还 batch 页给全局分配器，避免空闲页在 PCP 中积压导致其他 CPU 或高阶分配无法获取。

```rust
pub fn dealloc_page_cached(&self, cpu_id: usize, cpu_count: usize,
                           page: usize, kind: UsageKind) {
    let slot = cpu_id % PCP_CPU_SLOTS;
    let _preempt_guard = NoPreemptIrqSave::new();  // 关闭抢占
    let mut cache = self.page_caches[slot].lock();
    cache.release(self, total_pages, cpu_count, page, kind);
    // release 内部：push 入链表；若 count >= high，调用 drain 批量退页
}
```

全局分配失败（如剩余内存不足或高阶连续页无法满足）时，慢路径会先调用 `drain_page_caches()` 把所有 CPU 的 PCP 缓存全部退回全局分配器再重试一次，既防止空闲页被藏在 PCP 中造成的“假 OOM”，也为高阶连续页合并创造了机会。

3.2 优化二：AddrSpace 三段式 —— “prepare/execute/commit”

3.2.1 设计思路

如 2.4 根因所述，缺页在持有整个进程 `AddrSpace` 独占锁期间执行分配、页缓存、ext4 与磁盘 I/O，一个线程等盘时其他缺页、`mmap`、`munmap`、`mprotect` 全部排队（mutex 等待累计 3369.74 s，约 89% 经缺页路径）。因此我们把缺页处理拆成 prepare / execute / commit 三个阶段，将慢操作全部移到 `AddrSpace` 锁外：



图 7　AddrSpace 三段式缺页处理流程

- **prepare**：持有读锁，只做判断，确定要处理什么——定位 VMA、校验权限、检查 PTE 是否已被其他线程装好、构造 `FaultPreparation`/fault plan，不读盘、不分配；
- **execute**：不持有 `AddrSpace` 锁，并发执行一些慢操作：物理页分配、页缓存查找/创建、等待 folio loader、进入 ext4、等待磁盘、COW 数据复制等；
- **commit**：重新取得读锁，再次定位 VMA 并校验权限，确认 prepare 时的前提仍成立后用原子 CAS 安装 PTE，同时登记 PTE 对应的 frame/folio 所有权。

提交结果分为 `Handled`（本线程成功或目标已被正确处理）、`Stale`（前提过期，需要重试）与错误三类。实现用 `MAX_CONCURRENT_FAULT_RETRIES = 4` 限制乐观重试次数，连续 stale 后回退到拿写锁的旧慢路径，保证极端竞争下仍能推进。

3.2.2 实现要点

prepare 阶段在 AddrSpace 读锁内完成判断，返回一个 FaultPreparation 枚举值；execute 阶段在锁外执行，完事后在 commit 阶段重新获取读锁、验证前提并原子提交，最终返回 Handled 或 Stale：

```
/// prepare 的结果：要么已就绪，要么带一个可锁外执行的 fault plan
pub(crate) enum FaultPreparation {
    Handled,                        // PTE 已被其他线程安装，无需处理
    Cow(CowFaultPlan),              // 私有页（匿名/COW 文件）缺页计划
    File(FileFaultPlan),            // 共享文件 folio 缺页计划
    Shared(SharedFaultPlan),        // 共享匿名页缺页计划
    Fallback,                       // 退回旧写锁慢路径
}

/// commit 的结果
pub(crate) enum FaultCommit { Handled, Stale }
```

核心调度逻辑在 handle_page_fault_concurrent_inner 中：循环内先拿读锁做 prepare，再在锁外 execute，最后拿读锁 commit；Stale 时有限重试，连续失败后退回 Fallback 走写锁慢路径：

```
fn handle_page_fault_concurrent_inner(aspace, vaddr, access_flags, ...) -> bool {
    let mut stale_retries = 0;
    loop {
        // ── Phase 1: prepare（持有 AddrSpace 读锁）──────────────
        let preparation = {
            let guard = aspace.read();
            // 校验地址、定位 VMA、检查权限
            area.backend().prepare_fault(vaddr, area_range, flags, access_flags,
&guard.pt)
        };  // 读锁在此释放

        // ── Phase 2: execute（不持有 AddrSpace 锁）──────────────
        match preparation {
            FaultPreparation::Handled => { flush_tlb; return true; }
            FaultPreparation::Fallback => break,  // 退回写锁慢路径
            FaultPreparation::Cow(mut plan) => {
                plan.execute()?;  // 锁外：分配页、读盘、COW 复制
                // ── Phase 3: commit（重新获取读锁）──────────
                let guard = aspace.read();
                // 重新定位 VMA、重新校验权限
                area.backend().commit_fault(&mut plan, ...)
                // commit 内部：CAS 安装 PTE，登记 frame 所有权
            }
            // File / Shared 分支同理...
        };
        match committed {
            FaultCommit::Handled => { flush_tlb; return true; }
            FaultCommit::Stale => { stale_retries += 1; continue; }  // 有限重试
        }
    }
    // Fallback：拿写锁走旧慢路径，保证极端竞争下仍能推进
```

```
    aspace.write().handle_page_fault(vaddr, access_flags)
}
```

prepare 和 commit 都使用读锁而非写锁，原因是 VMA 树的结构性修改（mmap/munmap/改变 VMA 的 mprotect）仍走写侧，而缺页线程只读 VMA、不改变树结构，PTE 的并发安装由更细粒度的原子 CAS 协调。

3.3 优化三：Folio 系列：folio 页缓存与只读文件页直接映射

Folio 系列同时解决两个根因：页缓存粒度小导致的逐页查找与重复 I/O（根因四），以及只读 MAP_PRIVATE 缺页多一次私有页分配与复制（根因三）。

3.3.1 folio 页缓存：批量加载与并发等待

Folio 这个做法参考了 Linux 的部分实现。将文件页缓存的管理单位从单页提升为 folio（一组连续的 4 KiB 基础页，最大 order = 4，即一个 folio 最多覆盖 16 页共 64 KiB）。用户页表仍按 4 KiB 安装 PTE，folio 只改变页缓存的查找、I/O、状态管理与回收粒度。

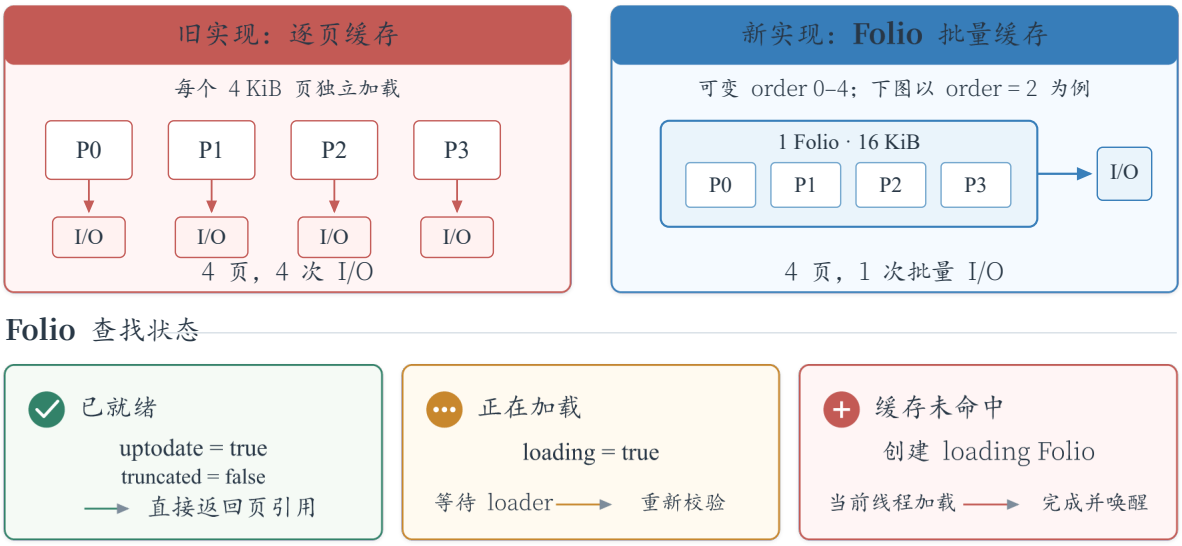


图 8　Folio 页缓存：单页 vs 批量加载，以及查找 Folio 的三种状态

每个 Folio 内部维护一个 FolioState，核心字段为 loading、uptodate、truncated 三个布尔标志，分别表示”正在从磁盘加载”、”数据已有效”、”已被截断失效”。查找 folio 时根据这三个字段走三条路径：已就绪则直接返回引用（最快，仅持每页锁）；正在加载则释放锁后等待 loader 完成再重新校验；缓存未命中则创建新 folio 并置 loading = true，由本线程作为唯一的 loader 发起磁盘 I/O，完成后置 uptodate = true 并唤醒所有等待者。

```
/// 每个 Folio 自身的生命周期状态
struct FolioState {
    loading: bool,          // 正在从磁盘加载数据
    uptodate: bool,         // 数据已有效，可以安全读取
    error: Option<VfsError>, // 加载失败时记录错误原因
    dirty: u16,             // 位图：哪些子页被修改过
    writeback: u16,         // 位图：哪些子页正在回写
    truncated: bool,        // 文件已被截断，缓存失效
}

pub struct Folio {
```

```
    index: u32,             // 起始页号（自然对齐到 order）
    order: u8,              // 2^order 页，最大 4（16 页 = 64 KiB）
    frame: FolioFrame,      // 连续物理页
    state: Mutex<FolioState>,
    page_locks: [Mutex<()>; MAX_FOLIO_PAGES], // 每子页一把锁
    wait: WaitQueue,        // 等待 loading 完成的队列
    // ...
}

// loader 完成 I/O 后调用：更新状态并唤醒所有等待者
fn complete_load(&self, result: VfsResult<()>) {
    let mut state = self.state.lock();
    state.loading = false;                    // 加载结束
    if result.is_ok() && !state.truncated { state.uptodate = true; }
    else { state.error = Some(...); }
    drop(state);
    self.wait.notify_all(false);              // 唤醒所有 waiter
}

// 等待方调用：阻塞直到 loading 变为 false
fn wait_uptodate(&self) -> VfsResult<()> {
    self.wait.wait_until(|| !self.state.lock().loading);
    // 醒来后检查 uptodate / error / truncated
}
```

页缓存索引按文件分成 64 个 shard，不同文件的缺页线程争抢不同 shard 的锁；同一 folio 只允许一个 loader 真正读盘，其他线程找到 `loading` folio 后等待其完成，避免重复读盘。用 `FolioPageRef` 保证已装入 PTE 的物理页在页缓存回收期间不会被提前释放。等待 loading folio 时必须先释放 `invalidate_lock`，完成后再重新加锁并检查同一页号是否仍是同一个 folio（因为等待期间 truncate 可能已替换），避免三方死锁。folio 并非越大越好，保留单页 fallback。

```
/// 每个文件的 folio 索引：64 个 shard 降低锁竞争
struct FileAddressSpace {
    folio_index: [RwLock<BTreeMap<u32, Arc<Folio>>>; 64],
    invalidate_lock: SleepRwLock,  // truncate 等破坏性操作持写侧
    // ...
}

/// 持有 Folio 的强引用，保证 PTE 指向的物理页不被回收
pub struct FolioPageRef {
    folio: Arc<Folio>,  // Arc 引用计数阻止 folio 被释放
    page: u8,           // 子页在 folio 内的偏移
}

/// 查找 folio 页的完整流程：释放锁等待 → 重新加锁 → 校验未变
fn with_page(&self, pn: u32, f: ...) -> R {
    // ① 持 invalidate 读锁，查 folio 索引
    let folio = {
        let _invalidate = self.invalidate_lock.read();
        self.lookup_folio(pn)
```

```
    };
    // ② 释放 invalidate 锁后再等待 loading（避免三方死锁）
    if folio.wait_uptodate().is_err() { return f(None); }
    // ③ 重新获取 invalidate 读锁，校验仍是同一个 folio
    let _invalidate = self.invalidate_lock.read();
    let current = self.lookup_folio(pn);
    if !Arc::ptr_eq(&current, &folio) { return f(None); }  // truncate 已替换
    // ④ 持每页锁 + 校验 uptodate/truncated → 安全访问
    let page_guard = folio.page_locks[page].lock();
    let state = folio.state.lock();
    if !state.uptodate || state.truncated { return f(None); }
    // ...
}
```

3.3.2 只读文件页直接映射（COW）

旧链路中，每次 `MAP_PRIVATE` 文件缺页的时候，即使只是读操作，也要经过“磁盘 → Page Cache → 分配私有页 + memcpy 4 KiB → PTE”的完整流程。大量只读文件页（`.rlib`、`.o`、源码）永远不被写，这些分配和复制全部浪费，且多进程映射同一文件时各复制一份，明显加剧了内存压力。

所以优化的核心思路是：对满足条件的只读缺页，直接把 folio 中已就绪的物理页以只读方式装入 PTE，跳过私有页分配和 memcpy。PTE 标记为只读以保证 `MAP_PRIVATE` 语义——进程真正写入时再次缺页（COW），此时才分配私有页、从 folio 复制内容、换为可写 PTE。这样，永远不被写的页零额外成本，被写的页仅在实际写入时付出一次复制代价。新旧链路对比如下：

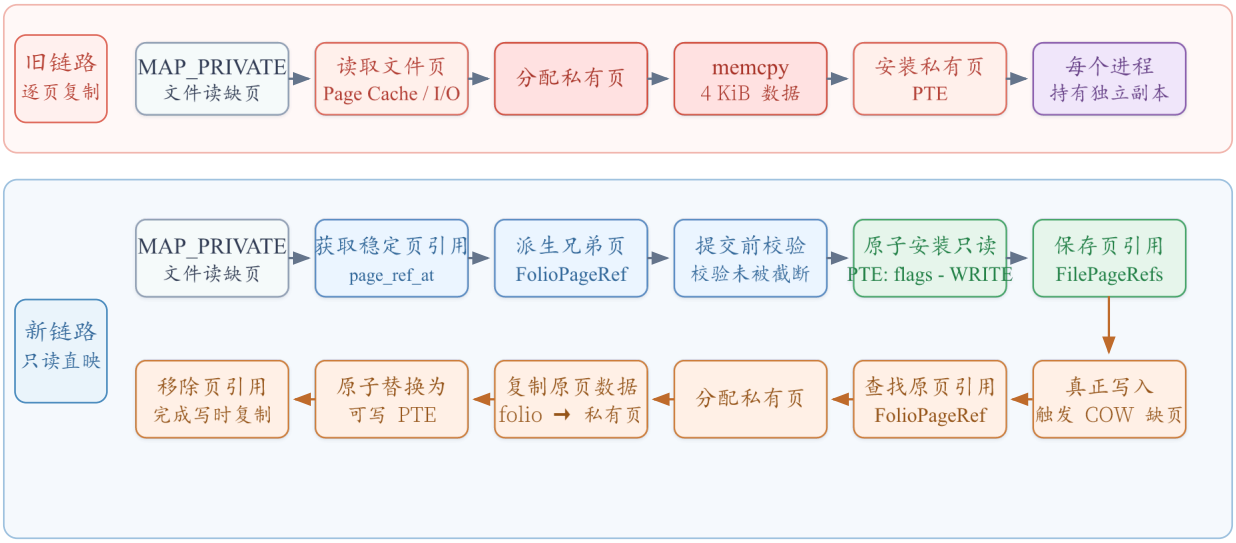


图 9　只读文件页直接映射：旧链路 vs 新链路

具体而言，execute 阶段在锁外调用 `file.page_ref_at(offset)` 获取 `FolioPageRef`——此时 I/O 已完成、folio 处于 uptodate 状态，拿到的是指向页缓存物理页的稳定引用。然后从 folio 派生出同一窗口内的兄弟页，一起交给 commit 阶段批量安装：

```
// CowFaultPlan::execute() —— 只读文件映射快路径（锁外）
if self.size == PageSize::Size4K
    && !self.access_flags.contains(MappingFlags::WRITE) // 非写缺页
    && start == 0 && max_read == self.size as usize      // 完整对齐页
{
    if let Some(fault_page) = file.page_ref_at(fault_file_start)? {
```

```
        // 从 fault folio 派生兄弟页，故障页优先
        self.file_pages.push((self.vaddr, fault_page.clone()));
        for addr in window_start..window_end {
            if addr != self.vaddr {
                if let Some(sibling) = fault_page.folio_page_ref(offset) {
                    self.file_pages.push((addr, sibling));
                }
            }
        }
    }
}
```

commit 阶段重新持读锁后，调用 `file.with_valid_folio_ref` 校验 folio 未被 truncate，然后以 `flags - MappingFlags::WRITE`（强制只读）CAS 安装 PTE，同时将 `FolioPageRef` 存入 `FilePageRefs` 表。后续真正写入触发 COW 时，从该表中查得原来的 `FolioPageRef`，据此分配私有页、复制内容、换为可写 PTE 并释放 folio 引用。

```
// CowBackend::commit_missing_fault() —— 安装只读 PTE
let validated = file.with_valid_folio_ref(&validation_page, || {
    for (addr, page) in pages {
        // CAS 安装 PTE，强制只读（flags - WRITE）
        match pt.map_4k_concurrent(addr, page.paddr(), flags - MappingFlags::WRITE) {
            Ok(()) => {
                file_pages.insert(addr, page);  // 保存 FolioPageRef，PTE 在位期间不释放
            }
            Err(PagingError::AlreadyMapped) => {}  // 并发已安装，跳过
            // ...
        }
    }
});
```

回退到旧慢路径的条件：写缺页、非 4 KiB 对齐、EOF 不完整尾页、页缓存无法返回稳定引用、commit 时 folio 已失效（truncated 或被替换）。

3.4 优化四：ext4 锁外数据读写 + VirtIO 多在途请求

如 2.4 根因所述，一次普通文件读取在 ext4 全局锁内要完成块映射查询并等待磁盘，跨文件 I/O 会被全局串行化。所以本优化分两层来解除串行：extent cache 与锁外数据 I/O（上层），异步 VirtIO 多在途请求（底层）。

3.4.1 extent cache 与锁外数据 I/O

先区分两类工作：extent 映射查询属于短元数据操作，数据 I/O 属于长等待操作。优化思路是把慢操作移出锁外——ext4 锁内只做“查 extent、记映射”，释放锁后再做磁盘 I/O。新旧读路径对比如下：

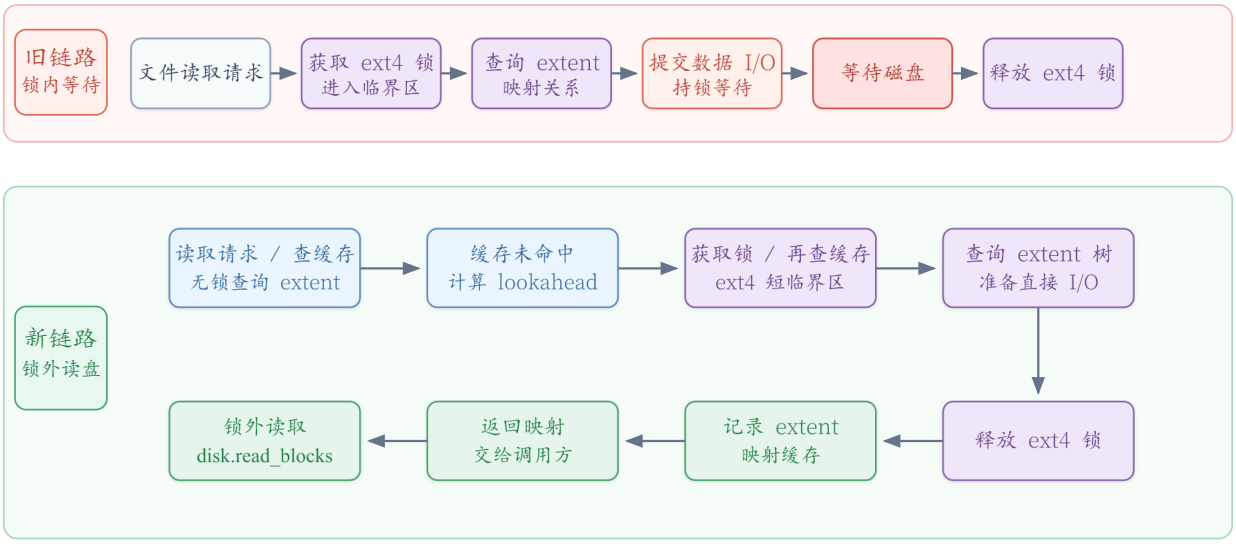


图 10　ext4 锁外数据 I/O：旧链路 vs 新链路

读路径上，extent cache 按 inode 分 64 个 shard，cache miss 时做约 1 MiB 的 lookahead 预取相邻 extent；进入 ext4 锁后必须再次检查 cache——因为排队等锁期间，其他线程可能已经填好了同一映射。稀疏文件的空洞区域没有分配物理块，直接填零返回。

```
fn read_extents(&self, ino, start, count, file_size) -> VfsResult<Vec<BlockExtent>> {
    // ① 先查 extent cache（无锁），命中则直接返回物理块映射
    if let Some(extents) = self.cached_extents(ino, start, count) { return
Ok(extents); }
    // ② 计算 lookahead 窗口（~1 MiB）
    let mapping_count = count.max(lookahead.min(available));
    let extents = {
        let mut fs = self.lock();     // ③ 短暂获取 ext4 锁
        // ④ 锁内再次检查 cache（排队期间可能已被其他线程填好）
        if let Some(extents) = self.cached_extents(ino, start, count) { return
Ok(extents); }
        let extents = fs.map_blocks(ino, start, mapping_count)?;  // 查 extent 树
        fs.prepare_direct_data_access(&extents)?;  // 准备直接 I/O
        extents
    };  // ⑤ 释放 ext4 锁
    // ⑥ 锁外：缓存 extent 映射，返回给调用方去做 disk.read_blocks
    self.cache_extents(ino, &extents);
    self.cached_extents(ino, start, count).ok_or(VfsError::Io)
}
```

写路径更复杂，因为写入可能涉及分配新块和更新文件长度，不能完全绕开 ext4：锁内分配或确认块映射并准备缓存一致性，锁外写数据块，最后再短暂进入 ext4 提交文件长度与元数据更新。文件中间非对齐部分块写仍走 lwext4 的 read-modify-write 安全路径。

```
fn write_mapped(&self, ino, buf, offset, file_len) -> VfsResult<usize> {
    self.invalidate_extent_cache(ino);  // 写操作先失效缓存，下次读重新查映射
    let (block_size, extents) = {
        let mut fs = self.lock();        // 获取 ext4 锁
        // 非对齐部分块写（修改块中间一小段）→ 回退 lwext4 安全路径
        if !offset.is_multiple_of(block_size)
```

```
            || (!buf.len().is_multiple_of(block_size) && write_end < file_len)
        { return fs.write_at(ino, buf, offset); }
        // 对齐整块写：锁内分配/确认块映射
        let extents = fs.allocate_blocks(ino, logical_block, block_count)?;
        (block_size, extents)
    };  // 释放 ext4 锁
    // 锁外写数据块 → disk.write_blocks(...)
    self.lock().commit_file_size(ino, new_size)?;  // 最后短暂入锁提交长度
}
```

缓存一致性方面，任何可能改变块分配的操作必须失效对应 inode 的 extent cache，否则旧映射会指向已被回收或重新分配的物理块。

```
/// 写操作、truncate、删除/重建 inode 前调用，使旧映射失效
pub(crate) fn invalidate_extent_cache(&self, ino: u32) {
    self.extent_cache_for(ino).write().remove(&ino);
}
```

此外有两项配合性改进。一是将 ext4 全局锁从自旋锁改为阻塞 Mutex：锁外 I/O 已经把持锁时间从“等磁盘的毫秒级”缩短为“查 extent 的微秒级”，此时再用自旋锁空转等锁已无必要，改为阻塞 Mutex 后等锁线程直接睡眠，把 CPU 让给其他编译任务。

二是脏页积累后批量写回：`CachedFile` 先将多个 folio 的脏页积累在内存中，汇总后再一次进入 ext4 提交文件长度和元数据更新，而不是每写回一个 folio 就获取一次 ext4 锁。这样减少了 ext4 锁的获取频率，避免数据 I/O 已经移出锁外、元数据提交又成为新的串行点。

3.4.2 底层块设备：异步 VirtIO 多在途请求

ext4 锁外 I/O 解决了上层串行化，但如果块设备层仍然一次只允许一个请求在途，不同文件的 I/O 还是被设备互斥锁串在一起。因此底层加入异步 VirtIO 块设备层，让多个 I/O 请求可以同时在 VirtIO 队列中等待设备处理。新旧链路对比如下：

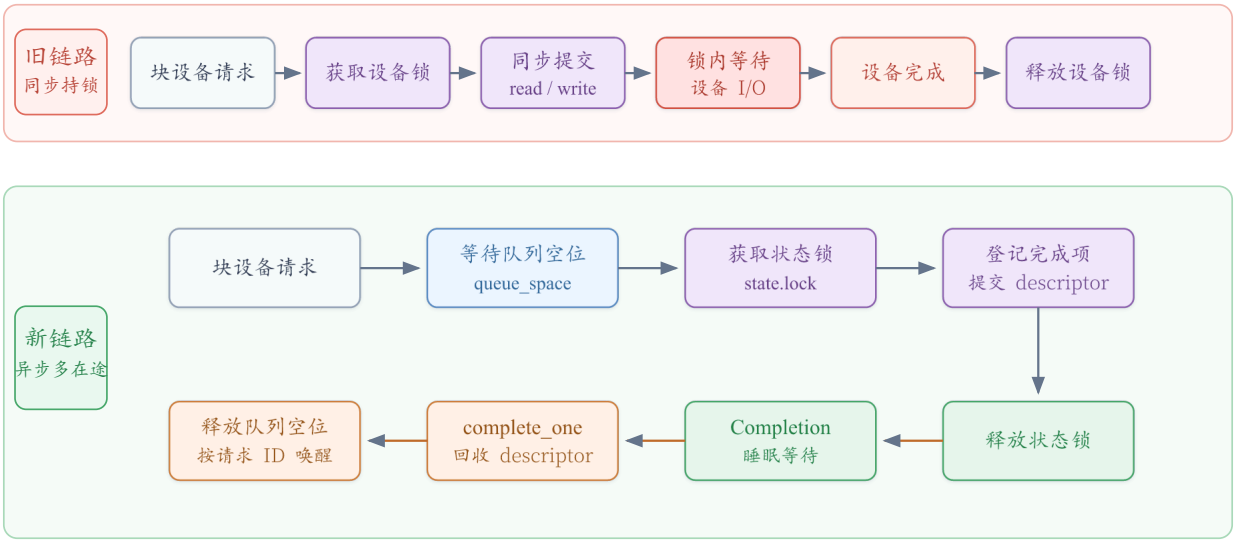


图 11　异步 VirtIO 块设备层：旧同步链路 vs 新异步链路

旧链路中，提交线程持设备锁完成”提交 → 等 I/O → 释放锁”的完整流程，一次只有一个请求在设备上。新链路中，设备锁只保护 descriptor 的提交与回收（微秒级），提交线程把请求放入 VirtIO 队列、通知设备后立即释放锁，然后在自己独立的 `Completion` 上睡眠等待。I/O 完成后由 completion

task 回收 descriptor 并按请求 ID 唤醒对应的提交线程。设备锁不覆盖磁盘完成时间，最多 16 个请求可同时在途。

核心数据结构为每请求一个 `Completion`，内含结果槽和等待队列：

```
struct Completion {
    result: Mutex<Option<DevResult>>,  // I/O 完成后由 completion task 填入
    wait: WaitQueue,                   // 提交线程在此睡眠等待
}

impl Completion {
    fn finish(&self, result: DevResult) {       // completion task 侧调用
        *self.result.lock() = Some(result);
        self.wait.notify_one(false);            // 唤醒提交线程
    }
    fn wait(&self) -> DevResult {               // 提交线程侧调用
        self.wait.wait_until(|| self.result.lock().is_some());
        self.result.lock().take().unwrap()
    }
}
```

提交路径：持设备状态锁 → 分配请求 ID → 调用 `submit_read/submit_write` 放入 VirtIO 队列 → 通知设备 → 释放状态锁 → `completion.wait()` 睡眠。队列满时在 `queue_space` 上等待，每完成一个请求唤醒一个提交者避免惊群。

```
unsafe fn submit(&self, write: bool, block_id, buffer) -> DevResult {
    // 系统启动早期中断未就绪 → 仍走同步 I/O
    if !ASYNC_BLOCK_IO_READY.load(Ordering::Acquire) {
        return self.submit_sync(write, block_id, buffer);
    }
    self.0.queue_space.wait_until(|| self.queue_has_space());  // 等队列有空位
    let id = self.0.next_request.fetch_add(1, Relaxed);
    let completion = Arc::new(Completion::new());
    {
        let mut state = self.0.state.lock();                    // 短暂持设备锁
        state.pending.insert(id, completion.clone());
        state.device.submit_read(id, block_id, buffer)?;      // 提交 descriptor
    }                                                             // 释放设备锁
    self.0.work.notify_one(false);              // 通知 completion task
    return completion.wait();                   // 睡眠等待 I/O 完成
}
```

`flush` 会先等待全部在途请求排空后再独占执行，可以保证文件系统一致性。

3.5 优化五：mprotect 按 VMA 定位并批量修改权限

`mprotect(addr, len, prot)` 的作用是修改当前进程 `[addr, addr+len)` 这段虚拟地址的访问权限，例如将一段内存从可读写改为只读。BuildStorm 中 rustc/LLVM 会频繁调用 `mprotect` 来翻转代码段和数据段的权限，调用量极大（预构建约 147 万次、正式阶段约 65 万次）。

旧实现维护了一个全局的 `MMAP_RECORDS` 数组，混合保存所有进程的映射记录。一次 `mprotect` 需要锁住该全局数组，扫描全部记录以检查 seal 和最大权限，修改页表后再次锁住并重建整个数组。平均一次调用扫描约 3.16 万条记录，与当前进程实际涉及的区间大小完全无关。新旧链路对比如下：

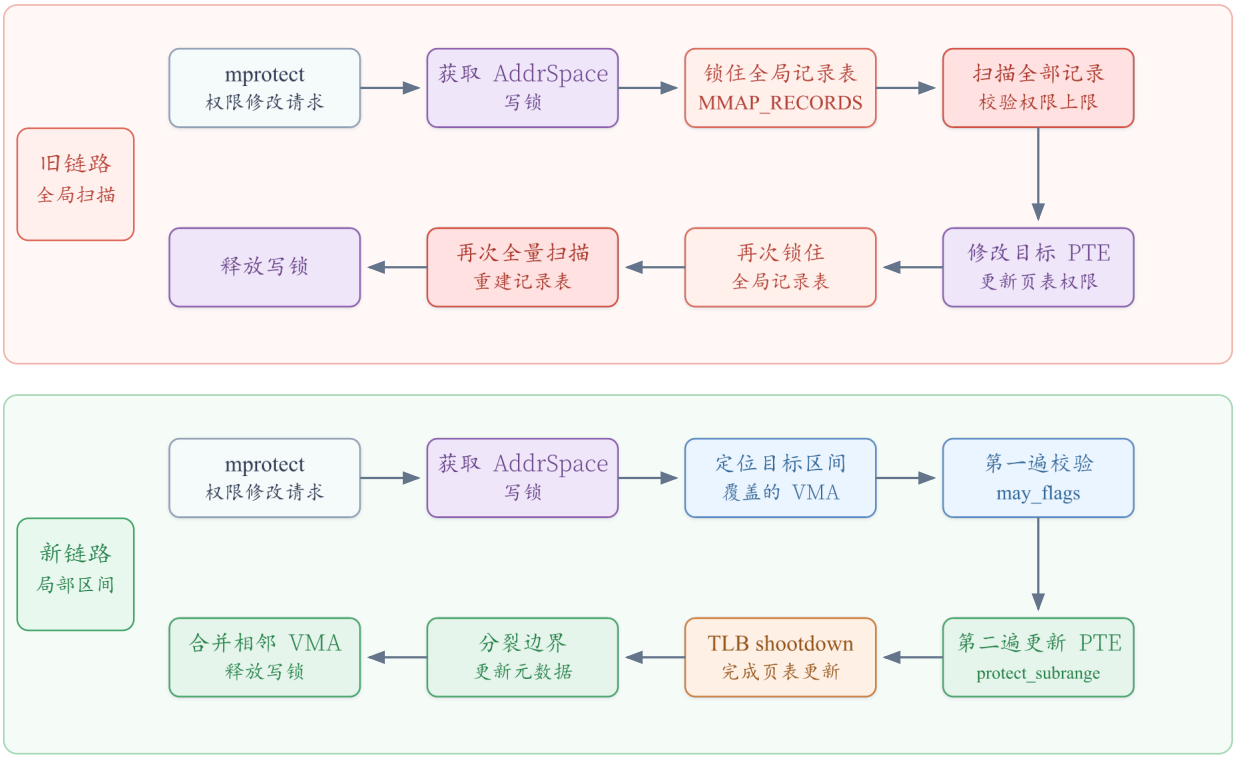


图 12　mprotect 按 VMA 定位：旧全局扫描 vs 新局部区间

优化的核心思路是删除全局 `MMAP_RECORDS`，把权限元数据直接嵌入每个 VMA。每个 `MemoryArea` 携带一个 `VmaMetadata`，其中 `may_flags` 记录了该 VMA 允许的最高权限——`mprotect` 只需检查目标区间的几个 VMA 是否在 `may_flags` 允许范围内即可，不用再扫描全局记录。

```
/// 嵌入每个 VMA 的权限元数据，替代全局 MMAP_RECORDS
pub struct VmaMetadata {
    user_flags: MappingFlags,  // 用户当前看到的权限
    may_flags: MappingFlags,   // 允许提升到的最大权限（seal / 只读文件限制）
    shared: bool,
    file_backed: bool,
    anonymous: bool,
    locked: bool,
}
```

`mprotect` 必须保证“要么全部成功，要么全部失败”，因此采用两遍式。第一遍只校验不修改：cursor 沿 `[addr, addr+len)` 前进，每次用 `range(..=cursor).next_back()` 定位覆盖 cursor 的最后一个 VMA，检查 `may_flags` 是否允许目标权限；若目标权限与当前一致则标记 no-op。复杂度 `O(log N + K)`，仅访问实际覆盖的 K 个 VMA。

第二遍才实际修改：对每个受影响的 VMA，调用 `protect_subrange` 更新相交区间的 PTE 权限（仅修改实际需要变化的子范围），必要时按左右边界分裂 VMA，更新目标段 flags/元数据后，将兼容的相邻 VMA 合并，避免了地址空间碎片化。

```
// MemorySet::protect —— 两遍式 VMA 区间修改
fn protect(&mut self, start, size, update_flags, can_merge, pt) -> Result {
    // 第一遍：纯校验，不改任何数据
    let mut cursor = start;
    while cursor < end {
        let area = self.find(cursor)?;          // O(log N) 定位
```

```
        if update_flags(area)?.is_some() { needs_update = true; }
        cursor = area.end().min(end);
    }
    if !needs_update { return Ok(no_op); }      // 权限已一致，直接返回

    // 第二遍：修改 PTE → 分裂 VMA → 更新元数据 → 合并相邻 VMA
    cursor = start;
    while cursor < end {
        let area = self.find_mut(cursor)?;
        area.protect_subrange(cursor, len, new_flags, pt)?;  // 只改 PTE 子范围
        if area.start() < cursor { split_left(); }           // 按左边界分裂
        if area.end() > end { split_right(); }               // 按右边界分裂
        area.update_flags_and_meta(new_flags);
        merge_if_compatible(&mut self.areas);                // 合并相邻兼容 VMA
    }
}
```

但是还可能出现一种问题：当 mprotect 收紧权限（如可写改成只读）时，修改 PTE 只是第一步，但是其他 CPU 的 TLB 中仍缓存着旧的可写映射，如果此时就释放物理页，远端 CPU 就还可能通过旧 TLB 写入已被回收的页面。 所以多核 TLB 同步方面要遵守一定的释放顺序：

```
先修改 PTE
        -> 向所有相关 CPU 发送 TLB shootdown（RISC-V 通过 SBI RFENCE），确保旧映射在全部核心上失效
        -> 最后才能释放 frame/folio 引用
```

全程持有 AddrSpace 写锁，保证 VMA 树与页表不被并发的结构性修改打断。

3.6 优化六：地址空间回收与 TLB 刷新

前五项核心优化把文件缺页与内存管理主链路缩短后，进程退出和 exec 的地址空间清理与 TLB 同步成为新的可见热点。BuildStorm 上百个 rustc/cc1 进程频繁 fork/exec/exit，每次退出都要销毁一个包含大量映射的地址空间——旧实现中这个销毁过程既慢又阻塞前台线程，且每释放一页就触发一次全核 TLB shootdown。

旧链路：退出线程持有 AddrSpace 写锁 → 逐 area → 逐 4 KiB 页遍历整个虚拟地址范围（不论是否实际映射过）→ 每页 pt.unmap + pt.finish（每页一次跨核 TLB 刷新）→ 逐页查 frame refcount 表并释放 → 最后释放锁。旧链路中多次“逐页”是核心问题——既逐页扫、也逐页刷 TLB、还要逐页释放。总览新旧链路对比如下：

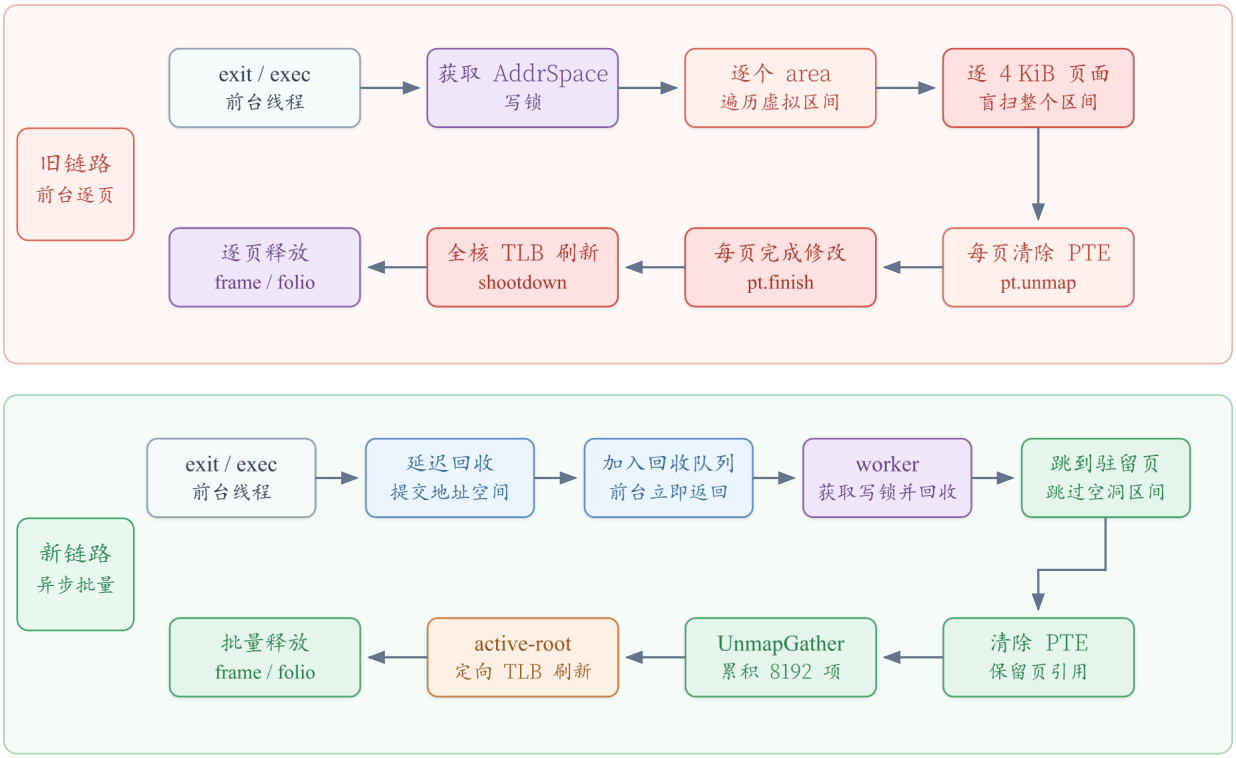


图 13　地址空间回收：旧同步逐页 vs 新异步批量 + 定向 TLB

优化的核心思路具体是四个层面的改进。

其一，异步回收。退出/exec 的前台线程通过 `defer_address_space_reclaim` 把已死亡的 `AddrSpace` 放入回收队列后立即返回，不再阻塞前台。后台有多个回收 worker 独立从队列中取出地址空间并执行 `clear()`，退出线程不再持有地址空间写锁等待清理完成。

```
pub(crate) fn defer_address_space_reclaim(aspace: Arc<AddrSpaceLock>, ...) {
    RECLAIM_QUEUE.lock().push_back(AddrSpaceReclaim { aspace, owner });
    start_reclaim_worker();  // 按需启动 worker，最多 cpu_num 个
}
```

其二，仅扫驻留页。旧实现用 `pages_in(range)` 逐 4 KiB 推进，遍历整个 VMA 范围的全部虚拟地址。新实现中 `unmap_for_clear` 使用页表自身的 `next_present_mapping(cursor, end)` 接口，沿页表树直接跳到下一个存在 PTE 的地址，整段空的中间页表层级被一次性跳过，不再逐页探测空洞。

其三，UnmapGather 合并 TLB 刷新。旧实现中每个 PTE 的 `pt.unmap` 后紧跟 `pt.finish()`，一个进程退出触发成千上万次独立的跨核 TLB 刷新。新实现中，`UnmapGather` 暂存被清除的 frame/folio 引用（匿名页的物理地址或文件页的 `FolioPageRef`），累计最多 8192 个驻留映射后才执行一次统一的 TLB shootdown，完成后一次性释放这批所有权。顺序是先清 PTE → 保留引用 → 完成远端 TLB 失效 → 再释放物理页，不可颠倒。

```
pub(crate) fn clear(&mut self) {
    let mut gather = UnmapGather::new();
    for area in areas.iter() {
        // unmap_for_clear: 沿 next_present_mapping 前进，每页清 PTE，
        // 将 frame/folio 引用 push 进 gather，满 8192 就先 shootdown 再释放
        area.backend().unmap_for_clear(area.va_range(), &mut pt, &mut gather)?;
    }
    gather.finish(&mut pt);  // 最后一轮 shootdown，释放剩余所有权
}
```

```
struct UnmapGather { pages: Vec<DeferredUnmapPage>; }  // 最多 8192 项

impl UnmapGather {
    fn finish(&mut self, pt: &mut PageTableMut) {
        pt.finish();  // 一次 TLB shootdown，覆盖本轮所有清过的 PTE
        for page in self.pages.drain(..) {
            match page {
                DeferredUnmapPage::Anonymous { paddr, page_size } =>
drop_referenced_frame(paddr, page_size),
                DeferredUnmapPage::File(page) => drop(page),  // 释放 FolioPageRef
            }
        }
    }
}
```

其四，定向 TLB 刷新（active-root）。这是基于前面合并刷新 TLB 基础上的进一步优化。旧实现中任何页表修改都通过 `pt.finish()` 无条件向全部 CPU 发送 TLB 刷新，即使大部分 CPU 根本没运行过这个地址空间。新实现中每个 CPU 在切换用户页表根之前，通过原子数组 `ACTIVE_PAGE_TABLE_ROOTS` 发布当前页表根的物理地址。TLB shootdown 时根据 root 筛选：RISC-V 只向仍运行该地址空间的 hart 发 SBI `remote_sfence_vma`，LoongArch 只向选中 CPU 发 IPI 执行按 VA 的 `invtlb`。root 为 0 或无法确定时保守刷新所有 CPU。CPU 切入新 root 时的顺序是先发布 root、再安装页表、随后本地全刷，避免与并发 shootdown 竞态时漏掉目标 CPU。

```
// 每个 CPU 发布当前用户页表根
static ACTIVE_PAGE_TABLE_ROOTS: [AtomicUsize; CPU_NUM] = ...;

// RISC-V: 仅向运行目标地址空间的 hart 发送远程 TLB 刷新
let mask = cpu_bits.iter().enumerate().filter(|(cpu, _)| {
    ACTIVE_PAGE_TABLE_ROOTS[cpu].load(Acquire) == root
});
sbi_rt::remote_sfence_vma(HartMask::from_mask_base(mask, base), start, size);
```

3.7 其他优化

核心优化完成后，缺页提交路径上的引用计数、PTE 安装与 VFS 名字解析/元数据读取成为剩余热点。以下优化属于细碎改进：frame 引用计数直查、PTE/COW/Folio 快速提交，以及 VFS 目录项与 inode 元数据并发缓存。

3.7.1 缺页提交微路径

核心缺页链路缩短后，引用计数查询、PTE 提交和 COW 所有权记录开始出现在火焰图中。以下四项微优化各针对一条局部热点，按”旧做法 → 问题 → 新做法”分述。

frame 引用计数直查。旧实现将每张物理页的引用计数存入 `BTreeMap<PhysAddr, Arc<SpinNoIrq<FrameRefCnt>>>`，分配、COW、释放时都需要 BTree 查找且全程关中断。新实现根据物理内存的起始地址和页号计算紧凑数组下标——`(paddr - base) / 4KiB` 直接定位到 64 路分片的槽位，O(1) 查表。同时将临界区从”关中断”缩短为”仅禁止抢占”，因为普通引用增减不涉及中断上下文，只有 PTE 安装/移除的原子 CAS 侧才需要更重的同步。

```
// 新：紧凑数组，物理地址 → 直接计算下标
struct FrameRefTable {
```

```
    shards: [SpinPreempt<BTreeMap<usize, FrameRefState>>; 64],
}
fn location(paddr: PhysAddr) -> FrameRefLocation {
    let offset = paddr.as_usize() - base.as_usize();
    FrameRefLocation { shard: (offset / 4096) % 64, index: offset / 4096 }
}
```

PTE commit 删除重复查询。旧实现中 missing-fault commit 先调用 `pt.query` 完整走一次多级页表遍历确认槽位为空，再执行 `pt.map` 做第二次页表遍历并 CAS 安装。新实现直接调用 `pt.map_4k_concurrent`，CAS 返回的旧 PTE 值即可区分三种情况：空槽（本线程安装成功）→ `Handled`；已存在等价映射 → `Handled`（竞争已处理）；状态不满足 → `Stale` 走有限重试。删掉一次完整页表遍历。

COW 文件页内联槽。`MAP_PRIVATE` 文件映射只读直接安装 folio 页时，必须记录”这个 PTE 正在固定哪个 `FolioPageRef`”，以便后续 COW 写缺页时查得原引用并替换。旧实现无论多少文件页一律进 BTree。新实现优先使用 backend 内的一个内联槽（仅一个条目），超出容量才升为 BTree overflow；删除内联项后从 overflow 提升一项填补。纯匿名映射根本不创建文件页所有权表。

folio fault-around。从故障页所在的 folio 派生同窗口内的兄弟页，一次 `with_valid_folio_ref` 校验整个 folio 的有效性后，循环内按地址由低到高逐一 CAS 安装多张 4 KiB PTE，最多覆盖一个 folio（16 页）。故障页优先安装保证原始缺页先被满足，兄弟页冲突可跳过不影响整体。相对当时的正确性基线，BuildStorm 单轮改善约 7.44%。

3.7.2 VFS 目录项与 inode 元数据并发缓存

BuildStorm 不仅密集读取文件内容，还频繁执行 `lookup`、`stat`、创建临时文件和重命名。ext4 数据 I/O 已经移出全局锁，但如果 VFS 层在做一次冷目录查询时持有目录缓存写锁等待磁盘，其他线程的缓存命中仍会被连带阻塞。

目录缓存方面，旧实现只缓存 positive dentry（已确认存在的名字），对不存在的名字每次都要进入 ext4 重复查询。新实现同时缓存 negative dentry，把不存在的名字也纳入缓存，构建系统反复探测同一不存在的路径时直接返回 `NotFound`。每个目录设置 16 个按文件名哈希的睡眠锁：同名并发 miss 只允许一个线程实际执行文件系统 lookup，其他等待者醒来后重新检查缓存；不同名字仍可并行。lookup 期间不持有目录 cache 写锁，查询完成后通过 `cache_generation` 检查期间是否发生 namespace 修改，只有代数一致才发布结果。create/unlink/link/rename 等修改操作由目录 `mutation_lock` 串行提交，跨目录 rename 按地址顺序取两把锁避免死锁。

inode 元数据缓存方面，旧实现用一把全局缓存锁保护所有 inode 的属性读取。新实现改为 64 路分片，同一 inode 的加载与回写使用睡眠锁合并（避免重复读取），不同 inode 的 `stat`/属性读取不再争同一把锁。

第 4 章 实验分析

4.1 实验环境与方法

实验均在本地机器上完成，RISC-V 64 架构，QEMU `virt` 平台、8 vCPU、8 GiB 内存，磁盘为仓库 `sdcard-rv.img` 配合 `-snapshot` 保证每次从相同状态启动。宿主机配置与测评机不同，因此本地数据主要用于横向比较各优化阶段的效果，绝对值可能与测评结果有偏差。宿主机配置：（待补充）。

从优化前基线开始，每完成一轮关键优化后均使用相同 profiler 配置重新采集火焰图，通过前后对比判断优化是否命中目标热点、瓶颈是否发生了迁移。正式性能数据使用关闭 guest profiler 的内核（`make kernel-rv`），配合宿主机每秒 `pidstat`/`mpstat`/`iostat` 采样。每个阶段只有一次正式样本，宿主机未做 CPU 隔离，阶段差值表述为“单次观测”。

4.2 性能数据展示

为方便展示，以下各表中阶段编号与优化措施的对应关系为：

```
00-U 为优化前基线；
01 为 per-CPU 物理页缓存（PCP）；
02 为 AddrSpace 三段式缺页；
03 为 folio 页缓存；
04 为只读文件页直接映射；
05 为 ext4 锁外数据 I/O 与异步 VirtIO；
06 为 per-VMA mprotect；
07-10 为其他优化（VFS 并发缓存、异步地址空间回收、active-root TLB、缺页提交微路径）的合并终点。
```

4.2.1 编译耗时

优化前基线正式 BuildStorm 编译耗时 1524.50 秒。六项核心优化完成后降至 628.53 秒，累计下降 58.77%，约 2.43 倍加速。后续其他优化完成后终点耗时 460.76 秒，相对基线累计下降 69.78%。

阶段	累积优化	耗时 (s)	相邻变化	相对基线
00-U	baseline	1524.50	—	—
01	+ PCP	1058.68	-30.56%	-30.56%
02	+ AddrSpace 三段式	1197.38	+13.10%	-21.46%
03	+ folio 页缓存	879.17	-26.58%	-42.33%
04	+ 只读文件直接映射	772.47	-12.14%	-49.33%
05	+ ext4 锁外 I/O + 异步 VirtIO	765.18	-0.94%	-49.81%
06	+ per-VMA mprotect	628.53	-17.86%	-58.77%
07-10	+ 其他优化	460.76	-26.69%	-69.78%

表 4-1　各阶段 BuildStorm 编译耗时

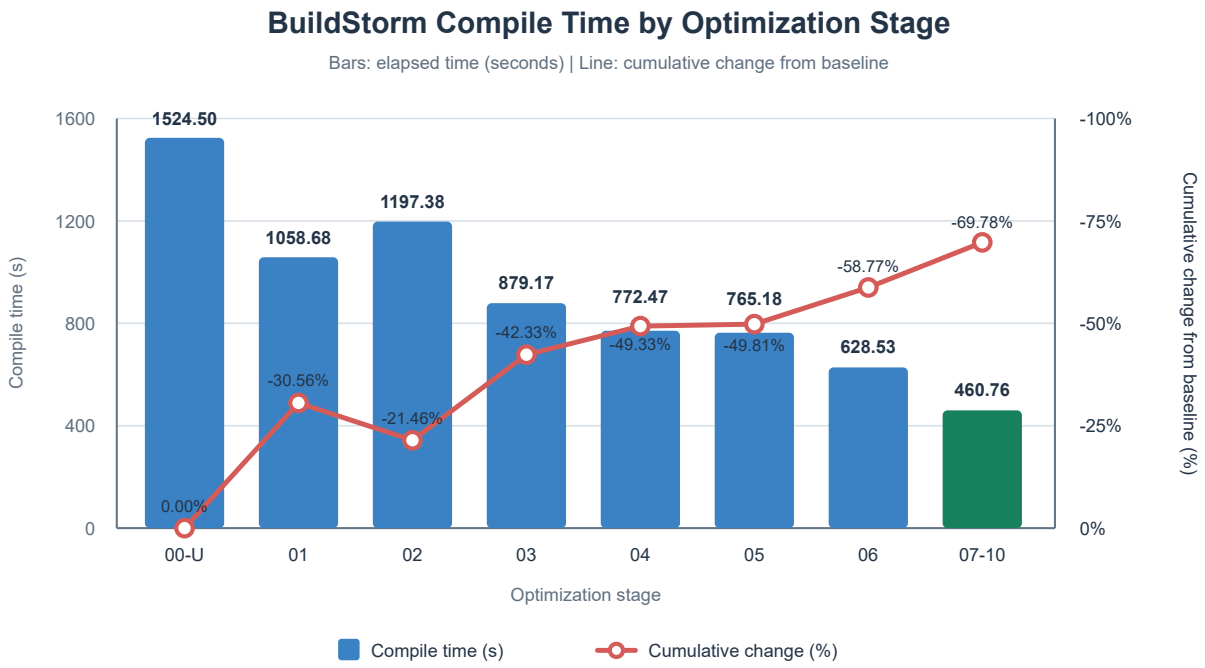


图 14　各阶段 BuildStorm 编译耗时与相对基线变化

4.2.2　CPU 并行度

把 QEMU 进程每秒 `%CPU / 100` 记为“有效核数”。核心趋势：平均有效核数从基线的 3.25 核逐步上升到 4.41 核，6～8 核高并行区间的时间占比从 6.23% 扩大到 42.93%，说明优化的主要效果不是让单核算得更快，而是让更多核能更长时间同时工作。阶段 07-10 为 dirty worktree 测量，未采集宿主 CPU 采样数据。

阶段	平均有效核数	中位数	**P95**	**6～8 核占比**
00-U	3.252	3.480	6.120	6.23%
01	3.536	3.080	7.341	22.38%
02	3.313	2.985	6.932	17.70%
03	3.598	2.050	7.770	32.61%
04	3.738	3.120	7.584	32.21%
05	3.725	3.030	7.605	31.46%
06	4.410	3.760	7.940	42.93%

表 4-2　各阶段 CPU 并行度

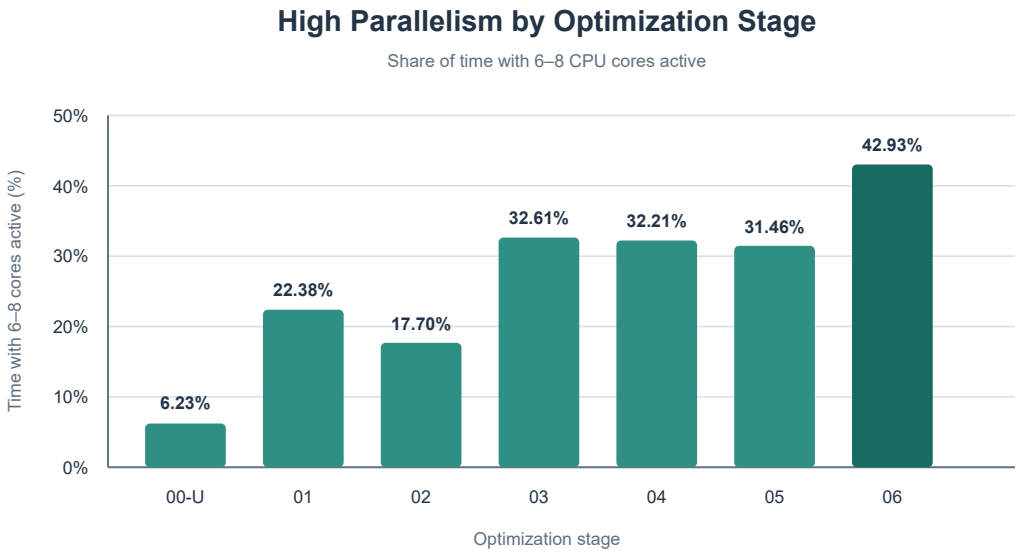


图 15　各阶段 6～8 核高并行区间时间占比

4.2.3 内存与 I/O

folio 阶段（03）用更多缓存与批量 I/O 换取性能，RSS 与读写量一度上升；只读直接映射（04）省掉私有页分配和复制后 RSS 明显回落；ext4 锁外路径（05）显著减少重复写入。最终版（06）相对基线：平均 RSS 基本持平（+0.51%），估算读取量 −14.61%，估算写入量 −36.87%。

阶段	平均 **RSS (KiB)**	最大 **RSS (KiB)**	估算读取 **(KiB)**	估算写入 **(KiB)**
00-U	2551119	2921592	469282	390434
01	2594330	3070044	523044	362744
02	2597466	3089976	448212	373526
03	3171058	3927916	639157	535631
04	2582383	3141188	523871	495356
05	2595280	3084728	403419	261508
06	2564150	3124756	400715	246463

表 4-3　各阶段内存与宿主机 I/O 估算（正式窗口）

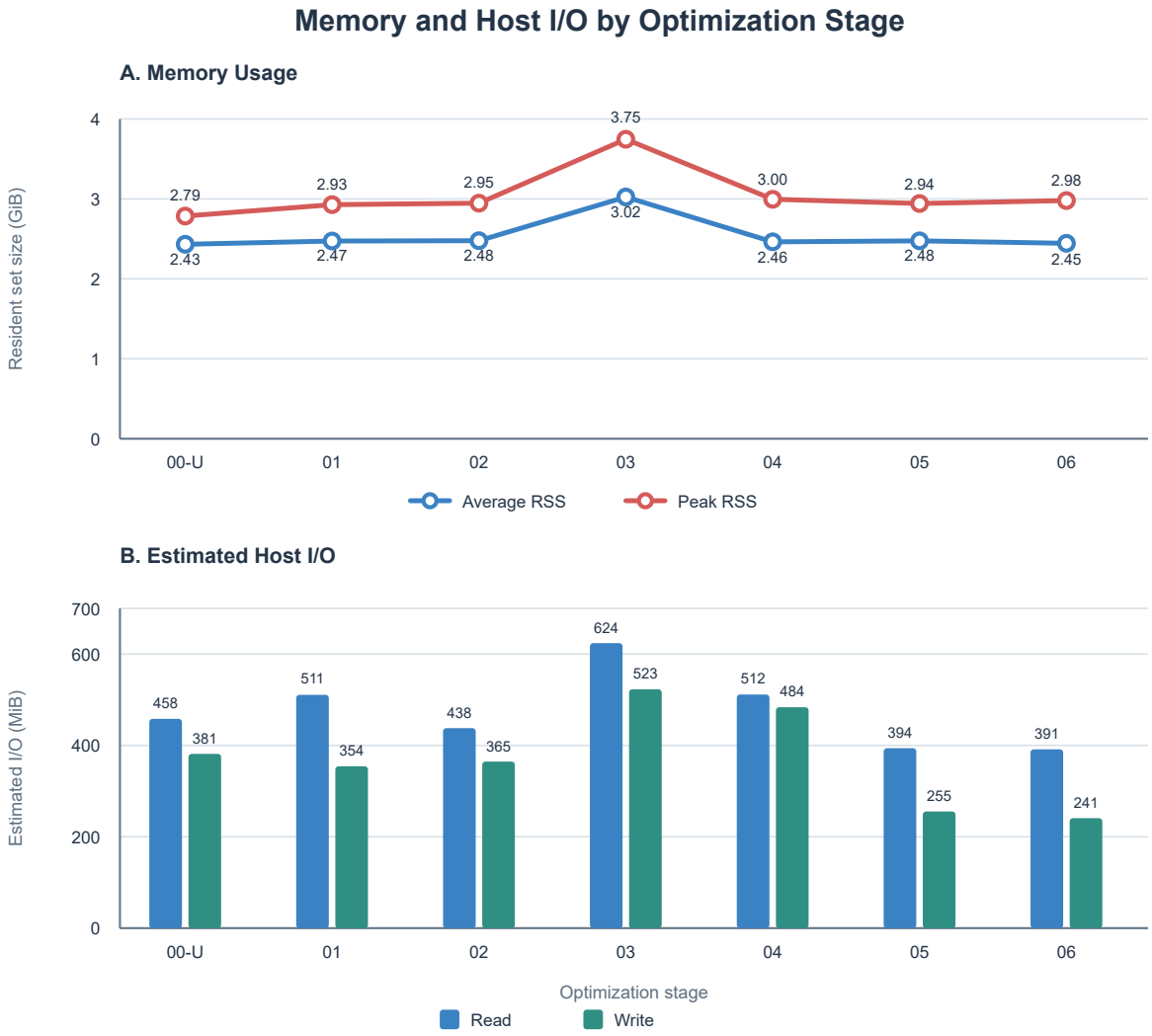


图 16　各阶段内存占用与宿主机 I/O 估算（上：平均与峰值 RSS；下：估算读取与写入）

上图反映的是各项优化对资源使用的影响，而非直接的墙钟收益。阶段 03 的 folio 页缓存通过扩大缓存粒度和批量 I/O 减少逐页缺页与等待，代价是 RSS 和读写量阶段性上升；阶段 04 的只读文件页直接映射省去了私有页分配与复制，使平均 RSS 相对前一阶段下降 18.56%，重新接近基线；阶段 05 将 ext4 数据 I/O 移出全局锁并引入异步 VirtIO 后，估算读取量和写入量相对前一阶段分别下降 22.99% 和 47.21%，说明重复 I/O 与串行等待得到缓解。到阶段 06，平均 RSS 相对基线仅增加 0.51%，峰值 RSS 增加约 6.95%，而估算读取量和写入量分别减少 14.61% 和 36.87%。

因此，这组单次观测表明：相关优化在显著缩短编译耗时、提高并行度的同时，没有造成持续性的平均内存膨胀，并降低了宿主机 I/O 压力。

4.3　逐项分析

4.3.1　PCP

相邻单次 −30.56%（1524.50 → 1058.68 s），平均有效核数由 3.25 升至 3.54，6～8 核占比由 6.23% 升至 22.38%。是本轮最强的单项端到端证据之一。

4.3.2　AddrSpace 三段式

相邻单次 +13.10%，该轮同时引入 COW commit 的远程 TLB flush 等新同步开销。三段式提供了后续并发框架，单轮数据不能证明独立收益。

4.3.3 folio

相邻单次 −26.58%，平均有效核数 +8.61%，代价平均 RSS +22.08%。用更多缓存与批量 I/O 换取更少逐页缺页与等待。

4.3.4 只读文件直接映射

相邻单次 −12.14%，平均 RSS −18.56%，宿主机 minor faults −84.71%。代价是 system CPU time 上升约 4 倍，直接推动了后续 UnmapGather 和 active-root TLB 优化。

4.3.5 ext4 锁外 I/O 与异步 VirtIO

相邻单次 −0.94%，端到端收益有限（宿主机 iowait 约 0.02%），但估算写入量 −47.21% 和读取量 −22.99% 表明 I/O 路径的串行化已被解除。

4.3.6 per-VMA mprotect

相邻单次 −17.86%，平均有效核数 +18.39%，6～8 核占比 +11.46 个百分点，总 CPU 时间仅降 2.58%——墙钟改善主要来自并行度提升。历史紧邻 A/B：旧全局 MMAP_RECORDS 路径 1243.89 s，per-VMA 实现 567.41 s（-54.38%）。

4.3.7 后续优化

Stage 08（异步回收 + UnmapGather）-16.39%，system CPU −87.66%。Stage 09（active-root TLB）-14.02%。Stage 10（微路径）-1.17%，已进入平台区。当前分支最低单次 504.26 s。

4.4 小结

同口径单次测量中，六项核心优化使 BuildStorm 从 1524.50 秒降至 628.53 秒（-58.77%），继续后续优化降至 460.76 秒（-69.78%）。平均有效核数从 3.25 升至 4.41，6～8 核占比从 6.23% 升至 42.93%。优化效果的核心体现不是单核更快，而是更多核能更长时间同时工作，与第 2 章定位的锁嵌套与锁粒度根因和第 3 章拆除锁链每一环的设计方向一致。

第 5 章 AI 使用说明与可复现步骤

5.1 AI 使用说明

本项目的开发过程使用了大型语言模型作为辅助工具，主要用于以下场景：

- 代码阅读与热点定位：让 AI 阅读内核源码与 profiler 汇总（火焰图、路径探针），协助梳理调用链，定位锁竞争与热点函数；
- 方案设计与代码生成：在人工确定优化方向和约束后，由 AI 生成关键优化代码（如 PCP 页缓存、三段式缺页框架、folio 状态机、per-VMA `mprotect`、extent cache 等），团队成员逐行 review 后合入；
- 正确性审查：AI 辅助检查 Rust 所有权/生命周期、锁顺序、引用计数与 TLB 失效顺序等容易出错的地方；
- 实验与数据处理：AI 辅助编写/检查实验脚本，解析 `console.log` 与 `host-summary.json`，生成阶段对比表与报告；
- 文档撰写：本报告由 AI 辅助整理实验数据与设计说明，所有数字均来自仓库中实际保存的实验记录，未采用 AI 推测的性能数值。

使用原则：所有 AI 生成的代码必须通过人工审查、单元测试与完整 BuildStorm 回归后才合入；涉及性能的结论只使用实际测量数据；失败的尝试也保留记录，避免重复踩坑。

5.2 可复现步骤

5.2.1 1. 环境准备

- 主机：x86_64 Linux，至少 8 个物理核；
- 工具链：Rust nightly-2025-05-20；`qemu-system-riscv64`；
- 磁盘镜像：仓库 `sdcard-rv.img`（评测镜像），运行参数与 `scripts/run_rv.sh` 一致，并使用 `-snapshot` 保证每次从相同状态启动。

5.2.2 2. 构建与运行

```
git switch experiment-data-organize
make kernel-rv                                  # 正式性能测量
make PROFILE=y kernel-rv                        # 热点定位
./scripts/run_rv_stage_metrics.sh docs/experiment_data/<stage>
```

5.2.3 3. 分阶段复现

从 baseline 按依赖顺序逐个应用提交并测量，每个阶段目录保存完整原始记录。阶段 07～10 的测量基点为 dirty worktree，实际代码差异保存在对应目录的 `source-worktree.patch` 中。

5.2.4 4. 正确性判定

每个阶段在统计性能前先确认：cagent 测试组与 minibuild 通过；`BUILDSTORM_COMPILE` 报告 `ok=true`；产物大小 1,681,000 字节；QEMU 正常退出且退出码为 0。

第 6 章 总结与展望

本文档完整记录了 MossOS 针对 BuildStorm 测例的问题定位、优化设计与实验验证过程。核心结论如下：

- 瓶颈定位明确：火焰图与探针数据交叉指向同一条热点链——缺页持有 AddrSpace 大锁进入页缓存和 ext4 等待磁盘 I/O，锁嵌套与锁粒度大两个结构性问题互相放大，导致 8 核沦为 2.36 核；
- 优化方向正确：六项核心优化按依赖顺序实施，逐一拆除“文件缺页 → 物理页分配 → 页缓存 → ext4 → 块设备 → mprotect”链路的锁嵌套和外层大临界区，后续优化继续缩短回收与 TLB 同步路径；
- 端到端效果显著：同口径单次测量中，BuildStorm 从 1524.50 秒降至 628.53 秒（六项核心优化，-58.77%），继续后续优化后降至 460.76 秒（累计 −69.78%）；平均有效核数从 3.25 升至 4.41，6～8 核占比从 6.23% 升至 42.93%；
- 证据分级透明：per-VMA `mprotect` 是证据最强的单项优化（紧邻 A/B −54.38%），folio、PCP、只读直接映射具备强单次正向证据；AddrSpace 三段式与 ext4 锁外 I/O 是架构必要机制，单轮端到端收益未充分证明。

在性能优化层面，当前工作仍有若干值得继续深入的方面。

现有优化主要围绕 BuildStorm 的读密集、顺序访问特征展开——folio 的 order 选择、fault-around 的窗口大小、PCP 的 batch 参数均基于该场景人工设定，缺乏对写密集或随机 I/O 负载的自适应能力。AddrSpace 三段式虽然提供了并发缺页的框架，但 COW commit 路径引入的远程 TLB flush 等新同步成本是否完全被 UnmapGather 和 active-root TLB 所抵消，尚未做严格定量拆解。部分微路径优化（PTE CAS 去重、frame 引用直查、COW 内联槽）能确认局部热点下降，但在 BuildStorm 端到端已进入平台区（约 504 秒）的背景下，更细粒度的改进需要更高分辨率的测量手段来验证。

mprotect 优化解决了 VMA 扫描的效率问题，但 mprotect 仍持有 AddrSpace 写锁，与 mmap/munmap 之间仍然串行——当前只是把锁内的工作量从 O(N_global) 降到了 O(K_local)，而没有解除锁本身的竞争。ext4 元数据操作（create/unlink/rename）仍走全局 ext4 锁，在大规模并行构建的临时文件创建场景中可能成为新的瓶颈。此外，当前优化仅在 RISC-V 平台上验证，LoongArch 上的 TLB 刷新语义、指令同步开销和 VirtIO 设备行为均有差异，优化效果与跨架构一致性有待确认。